

■ Error Detection Visualizer

A Web-Based Academic Tool for Understanding Error Detection Techniques

■ Overview

The Error Detection Visualizer is an interactive tool designed to help students understand how data transmission errors occur and how detection mechanisms such as Parity, Checksum, and CRC respond to those errors.

This project is part of the NIAT Training Module: **Applications in Computing — Error Detection Techniques**.

The visualizer demonstrates real-time encoding, transmission, error simulation, and detection workflows—making it ideal for classroom teaching and practical labs.

■ Table of Contents

- Overview
- Objectives
- System Architecture
- Encoding Techniques
- Error Simulation Features
- Receiver Module Functionality
- Interface Overview
- Use Cases
- Limitations
- Deployment
- Author

■ Objectives

This tool helps learners:

- Convert characters to ASCII and binary
- Generate redundancy bits (Parity / Checksum / CRC)
- Visualize bit errors during transmission
- Understand why some errors go undetected
- Learn how network protocols maintain data integrity

■ System Architecture

■ Processing Pipeline

Input Text → Sender Encoding → Transmission Frame → Error Simulation → Receiver Detection
→ Output Message

■ Major Components

1. Sender Module

- Converts text → ASCII → binary
- Generates parity bit / checksum / CRC
- Prepares transmission frame

2. Transmission Module

- Bit-stream visualization
- Shows payload and redundancy fields
- Allows bit-toggle interactions

3. Error Simulation Module

- Random bit flips
- Byte-specific corruption
- Manual bit editing
- Same-weight checksum-safe pairs

4. Receiver Module

- Recomputes detection values
- Matches with received frame
- Displays decoded output

■ Encoding Techniques

1. Even Parity

- Adds 1 parity bit per byte
- Detects odd bit flips
- Fails on even bit flips

2. Checksum (16-bit Internet Checksum)

Process:

- Break data into 16-bit words
- Add using wrap-around addition
- Take one's complement
- Append checksum

Behavior:

- Detects most random errors
- Can be bypassed using crafted 2-bit flips
- Visualizer demonstrates this weakness

3. CRC (Cyclic Redundancy Check)

Steps:

- Interpret data as polynomial
- Append zeros
- Perform polynomial division
- Append CRC remainder

Receiver:

Zero remainder → **No Error Detected**

CRC offers the strongest error detection capability.

■ Error Simulation Features

- ■ Random Bit Flip
- ■ Character-level corruption
- ■ Manual bit toggling
- ■ Same-weight checksum-safe flips
- ■ Auto-undetected checksum error generation

■ Receiver Module Functionality

The receiver performs:

- Parity verification
- Checksum recomputation
- CRC division and remainder check

Each technique displays:

- Calculated value
- Received value
- Pass/Fail detection result
- Final decoded message

■■■ Interface Overview

1. Sender View

- ASCII table
- Binary representation
- Parity/Checksum/CRC fields

2. Transmission View

- Bit-stream view
- Payload + redundancy bits
- Highlighted flipped bits

3. Receiver View

- Recomputed detection values
- Error status
- Final output text

4. Teacher Mode

- Diagnostic tools
- Internal computation view
- Editable received frame

■ Use Cases

- Teaching networking basics
- Demonstrating error detection weaknesses

- Comparing Parity vs Checksum vs CRC
- Understanding real-world corruption
- Lab activities for computing students

■ Limitations

- Source code intentionally hidden for academic security
- Build is obfuscated
- CRC animation is textual, not graphical

■ Deployment

- Deployed on Netlify
- Static HTML + JavaScript
- Works offline after first load
- No backend required

■■■ Author

Rishabh Shukla

NIAT Training Program — Error Detection Techniques